Portfolio Deployment with Firebase Hosting and GitHub Actions

1. Overview

This document explains how to deploy a React/Vite portfolio website to **Firebase Hosting**, connect a custom domain from **Hostinger**, enable HTTPS, and automate deployments using **GitHub Actions**.

The final architecture is:

```
Developer
   │
   │ git push
   ▼
GitHub Repository
   │
   │ GitHub Actions
   ▼
Install dependencies
   │
   ▼
npm run build
   │
   ▼
dist/
   │
   ▼
Firebase Hosting
   │
   ▼
malindasampath.com
```

After completing this setup, a normal deployment requires only:

```
git add .
git commit -m "feat: update portfolio"
git push
```

You no longer need to manually open Google Cloud Shell, pull the repository, build the project, or run `firebase deploy`.

---

2. Technologies Used

Technology	Purpose
React	Frontend framework
Vite	Frontend build tool
Git	Version control
GitHub	Source-code repository
GitHub Actions	CI/CD automation
Firebase Hosting	Website hosting
Google Cloud	Firebase project infrastructure
Hostinger	Domain/DNS management

---

3. Project Structure

The portfolio is a React/Vite application.

The important part of the project looks like this:

```
portfolio/
│
├── src/
│   ├── ...
│
├── public/
│   ├── ...
│
├── dist/
│   ├── index.html
│   ├── assets/
│   ├── images/
│   └── ...
│
├── package.json
├── .firebaserc
├── firebase.json
│
└── .github/
```

```
    └── workflows/
        ├── firebase-hosting-pull-request.yml
        └── firebase-hosting-merge.yml
```

The `dist` directory is the production build generated by Vite.

---

4. Create the Firebase Project

Go to the Firebase Console and create a Firebase project.

A Firebase project is also associated with a Google Cloud project.

In this setup, the project ID is:

```
project-c7396ba2-e46c-4c-17d7a
```

Firebase Hosting uses this project to store and serve the website.

---

5. Install Firebase CLI

Install the Firebase CLI on the development computer.

Verify the installation:

```
firebase --version
```

Example:

```
15.x.x
```

---

6. Authenticate with Firebase

Log in:

```
firebase login
```

If working in an environment where the normal browser login cannot be used:

```
firebase login --no-localhost
```

Complete the authentication using the Google account that owns the Firebase project.

Verify that the Firebase CLI can access the project.

---

7. Build the Portfolio

Because the portfolio uses Vite, the production website is generated using:

```
npm run build
```

This creates:

```
dist/
```

For example:

```
dist/
├── index.html
├── assets/
├── images/
├── icons/
└── ...
```

Firebase Hosting will serve the contents of this directory.

---

8. Initialize Firebase Hosting

From the portfolio project directory:

```
firebase init hosting
```

Select:

```
Use an existing project
```

Then select:

```
project-c7396ba2-e46c-4c-17d7a
```

When Firebase asks for the public directory:

```
dist
```

Because the application is a React SPA, answer:

```
Configure as a single-page app?
Yes
```

This creates:

```
firebase.json
```

and:

```
.firebaserc
```

---

9. firebase.json

The Firebase Hosting configuration points Firebase to the Vite build directory.

Example:

```
{
  "hosting": {
    "public": "dist",
```

```
    "ignore": [
      "firebase.json",
      "**/.*",
      "**/node_modules/**"
    ],
    "rewrites": [
      {
        "source": "**",
        "destination": "/index.html"
      }
    ]
  }
}
```

Important setting

```
"public": "dist"
```

means:

> Upload the files inside the `dist` directory to Firebase Hosting.

The rewrite configuration is important for React routing because Firebase needs to return `index.html` for routes that are handled by the React application.

---

10. Firebase Project Configuration

The `.firebaserc` file associates the local project with Firebase.

Example:

```
{
  "projects": {
    "default": "project-c7396ba2-e46c-4c-17d7a"
  }
}
```

This prevents Firebase from accidentally deploying to a different project.

---

11. First Manual Deployment

Before automating deployment, it is useful to verify that Firebase Hosting works manually.

Build the application:

```
npm run build
```

Then deploy:

```
firebase deploy --only hosting
```

Firebase will display a Hosting URL similar to:

```
https://project-c7396ba2-e46c-4c-17d7a.web.app
```

At this point the website is hosted by Firebase.

---

12. Connect the Custom Domain

The domain in this example is:

```
malindasampath.com
```

The domain is managed through Hostinger.

In Firebase Hosting:

1. Open Firebase Console.
2. Open the project.
3. Go to **Hosting**.
4. Select **Add custom domain**.
5. Enter:

```
malindasampath.com
```

Firebase provides DNS records that must be added to Hostinger.

---

13. Configure DNS in Hostinger

For the root domain, Firebase provided:

```
A    @    199.36.158.100
```

Firebase also provided a TXT verification record:

```
TXT    @    hosting-site=project-c7396ba2-e46c-4c-17d7a
```

The old A record pointing somewhere else must be removed.

For example, the old record was:

```
A    @    4.144.26.113
```

That old record must not remain because it would send visitors to the wrong server.

---

14. Verify DNS

DNS can be checked from Cloud Shell.

Check the A record:

```
nslookup -type=A malindasampath.com
```

Expected:

```
Address: 199.36.158.100
```

Check the TXT record:

```
nslookup -type=TXT malindasampath.com
```

Expected:

```
hosting-site=project-c7396ba2-e46c-4c-17d7a
```

Once Firebase detects these records, the domain becomes connected.

---

15. HTTPS / SSL Certificate

Firebase Hosting automatically provisions an SSL certificate for the custom domain.

Firebase may display:

```
Minting certificate
```

This can take some time.

Once the certificate has been successfully provisioned, the website is available through:

```
https://malindasampath.com
```

No manually purchased SSL certificate is required.

You can verify HTTPS from Cloud Shell:

```
curl -I https://malindasampath.com
```

A successful response looks like:

```
HTTP/2 200
```

The browser should then show the secure HTTPS connection.

---

16. Add www.malindasampath.com

The root domain:

```
malindasampath.com
```

and the `www` version are separate DNS/Hosting configurations.

In Firebase Hosting, add:

```
www.malindasampath.com
```

Firebase will provide the required DNS configuration.

The desired final result is:

```
https://malindasampath.com
https://www.malindasampath.com
```

Both should point to the portfolio.

---

17. Why Manual Deployment Is Not Ideal

Without CI/CD, every website update would require:

```
Make code changes
      ↓
Git push
      ↓
Open Google Cloud Shell
      ↓
Pull repository
      ↓
npm install
      ↓
npm run build
      ↓
firebase deploy
```

This is repetitive and unnecessary.

Instead, GitHub Actions can perform these steps automatically.

---

18. GitHub Repository

The source code is stored in:

```
malinda-sampath/portfolio
```

The repository contains the application's source code.

GitHub Actions watches this repository for changes.

---

19. Enable GitHub Actions Deployment

From the local portfolio directory:

```
firebase init hosting
```

Configure:

```
Public directory: dist
Single-page app: Yes
```

Then configure GitHub deployment:

```
firebase init hosting:github
```

Authenticate with GitHub.

Select:

```
malinda-sampath/portfolio
```

Firebase creates GitHub Actions workflows.

---

20. GitHub Actions Workflows

The setup creates:

```
.github/
└── workflows/
    ├── firebase-hosting-pull-request.yml
    └── firebase-hosting-merge.yml
```

These workflows automate Firebase deployments.

---

21. Pull Request Workflow

The pull-request workflow is used when changes are proposed through a Pull Request.

The general flow is:

```
Create Pull Request
        ↓
GitHub Actions starts
        ↓
Install dependencies
        ↓
Build application
        ↓
Deploy preview
        ↓
Review website
```

This allows changes to be tested before they become production changes.

---

22. Production Deployment Workflow

The merge workflow handles production deployment.

The flow is:

```
Developer
    │
    │ git push
    ▼
GitHub
    │
```

```
    ▼
Pull Request
    │
    ▼
Review
    │
    ▼
Merge to main
    │
    ▼
GitHub Actions
    │
    ├── npm ci
    │
    ├── npm run build
    │
    └── Firebase deploy
    │
    ▼
Firebase Hosting
    │
    ▼
malindasampath.com
```

---

23. Build Process in GitHub Actions

The workflow uses:

```
npm ci && npm run build
```

npm ci

Installs the exact dependencies defined by the project's lock file.

npm run build

Runs the Vite production build.

The result is:

```
dist/
```

Firebase then deploys the contents of `dist`.

---

24. GitHub Secret

Firebase creates a GitHub repository secret similar to:

```
FIREBASE_SERVICE_ACCOUNT_PROJECT_C7396BA2_E46C_4C_17D7A
```

This secret contains credentials used by GitHub Actions to authenticate with Google Cloud/Firebase.

It is stored in:

```
GitHub
→ Repository
→ Settings
→ Secrets and variables
→ Actions
```

Security rule

Never put the service-account JSON directly into:

```
Git
GitHub repository
firebase.json
Source code
```

It must remain a GitHub Secret.

---

25. Service Account

The GitHub Actions deployment uses a Google Cloud service account.

Firebase created:

```
github-action-1218307898@project-c7396ba2-e46c-4c-17d7a.iam.gserviceaccount.com
```

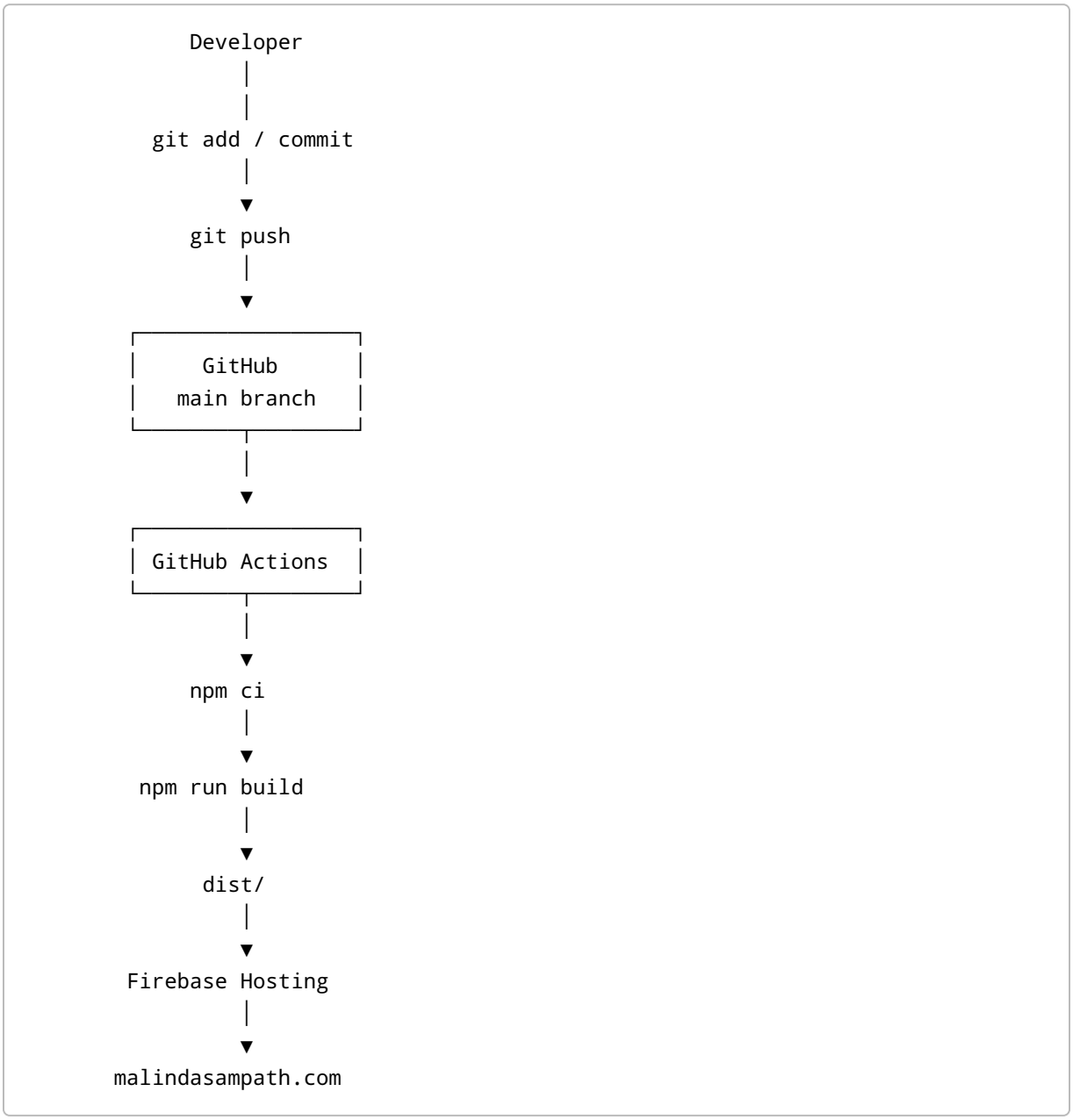
This account was given Firebase Hosting deployment permissions.

The service account allows GitHub Actions to authenticate and deploy the website without manually logging into Firebase.

---

26. The Final CI/CD Flow

The complete automated system is:



---

27. Normal Development Workflow

After everything is configured, daily deployment becomes very simple.

Make changes to the portfolio.

Then:

```
git add .
```

Commit the changes:

```
git commit -m "feat: update portfolio"
```

Push:

```
git push origin main
```

GitHub Actions automatically:

```
Detects push
    ↓
Installs dependencies
    ↓
Builds React/Vite application
    ↓
Deploys dist/
    ↓
Updates Firebase Hosting
```

No Cloud Shell is required.

No manual Firebase login is required.

No manual build is required.

No manual deployment is required.

---

28. Recommended Commit Messages

Use meaningful commit messages.

Examples:

```
git commit -m "feat: add new project to portfolio"
```

```
git commit -m "fix: correct mobile navigation"
```

```
git commit -m "style: improve project section layout"
```

```
git commit -m "docs: update portfolio content"
```

```
git commit -m "ci: update Firebase deployment workflow"
```

```
git commit -m "refactor: simplify portfolio components"
```

---

29. Checking Deployment Status

Open the GitHub repository and go to:

```
Actions
```

You can see:

```
Workflow
    ↓
Build
    ↓
Firebase deployment
    ↓
Success / Failure
```

A successful workflow means the new version has been deployed.

30. Common Problems

Firebase says project does not exist

Check the project ID:

```
firebase projects:list
```

Also check:

```
gcloud projects describe project-c7396ba2-e46c-4c-17d7a
```

Firebase Hosting API returns 403

Enable the Firebase Hosting API:

```
gcloud services enable firebasehosting.googleapis.com
  --project=project-c7396ba2-e46c-4c-17d7a
```

Verify:

```
gcloud services list --enabled
  --project=project-c7396ba2-e46c-4c-17d7a
  | grep firebasehosting
```

Expected:

```
firebasehosting.googleapis.com
```

Custom domain shows "Site Not Found"

Check:

1. Firebase Hosting deployment succeeded.
2. `dist` contains `index.html`.

3. DNS A record points to Firebase.
4. Old A records have been removed.
5. Firebase custom domain is connected.
6. SSL certificate has finished provisioning.

DNS changes are not detected

Check:

```
nslookup -type=A malindasampath.com
```

and:

```
nslookup -type=TXT malindasampath.com
```

DNS propagation can take time.

GitHub Actions build fails

Check the Actions logs.

First determine whether the failure occurs during:

```
npm ci
```

or:

```
npm run build
```

or:

```
Firebase deployment
```

Each stage has a different cause.

---

31. Important Files

The most important Firebase/CD files are:

```
firebase.json
```

Controls Firebase Hosting.

```
.firebaserc
```

Associates the local project with the Firebase project.

```
.github/workflows/firebase-hosting-pull-request.yml
```

Handles preview deployments for Pull Requests.

```
.github/workflows/firebase-hosting-merge.yml
```

Handles production deployment after changes are merged.

---

32. Final Result

The final website architecture is:

```
GitHub
   │
   │ Source Code
   ▼
GitHub Actions
   │
   │ CI/CD
   ▼
Firebase Hosting
   │
   │ HTTPS
   ▼
malindasampath.com
```

The domain is managed through Hostinger, while the actual website hosting is handled by Firebase Hosting.

The developer only needs to work with Git:

```
git add .
git commit -m "feat: update portfolio"
git push
```

Everything else is automated.

---

33. One-Line Summary

GitHub stores the code → GitHub Actions builds it → Firebase Hosting deploys it → Hostinger DNS points the domain to Firebase → Firebase provides HTTPS and serves the website.